Proposed Reduction Rules — Verification Notes

Mathematical Foundations for Implementation

Reference document for GitHub issues in the problem-reductions project

Abstract. This document formalizes nine reduction rules between NP-hard problems, providing complete construction algorithms, bidirectional correctness proofs, solution extraction procedures, and worked examples. Two reductions (§ 2) extend the NP-hardness proof chain from 3-SAT, potentially increasing the verified reachable count from 29 to 40+. The remaining seven (§ 3–5) resolve blockers in previously incomplete GitHub issues. Each entry is detailed enough to serve as a direct specification for implementation.

Contents

1 Notation and Conventions . . . 2
2 NP-Hardness Chain Extensions . . . 3
  2.1 SubsetSum → Partition . . . 3
  2.2 Minimum Vertex Cover → Hamiltonian Circuit . . . 4
  2.3 Vertex Cover → Hamiltonian Path . . . 6
3 Graph Reductions . . . 7
  3.1 MaxCut → Optimal Linear Arrangement . . . 7
  3.2 Optimal Linear Arrangement → Rooted Tree Arrangement . . . 9
4 Set and Domination Reductions . . . 11
  4.1 Dominating Set → Min-Max Multicenter . . . 11
  4.2 Dominating Set → Min-Sum Multicenter . . . 12
  4.3 Exact Cover by 3-Sets → Acyclic Partition . . . 13
5 Feedback Set Reductions . . . 15
  5.1 Vertex Cover → Partial Feedback Edge Set . . . 15
6 References . . . 18

1 Notation and Conventions

Throughout this document we use the following conventions:

- $G = (V, E)$ denotes an undirected graph with vertex set V and edge set E
- $n = |V|$, $m = |E|$
- $w : E \to \mathbb{Z}^+$ denotes positive integer edge weights (when weighted)
- $N[v] = \{v\} \cup \{u : (u, v) \in E\}$ is the closed neighbourhood of v
- For a set $S \subseteq V$, we write $w(S) = \sum_{v \in S} w(v)$

Each reduction entry contains:

1. **Theorem statement** — intuition and citation
2. **Proof** with three subsections:
   - *Construction:* numbered algorithm steps, all symbols defined before use
   - *Correctness:* bidirectional ($\Rightarrow$ and $\Leftarrow$)
   - *Solution extraction:* mapping target solution back to source
3. **Overhead table** — target size as a function of source size
4. **Worked example** — concrete instance with full verification

2 NP-Hardness Chain Extensions

2.1 SubsetSum → Partition

Theorem 2.1.1: Subset Sum reduces to Partition by adding at most one padding element. Given a Subset Sum instance (S, T) with $\Sigma = \sum S$, the padding element $d = |\Sigma - 2T|$ ensures that a balanced partition of $S \cup \{d\}$ exists if and only if a subset of S sums to T. This is the classical equivalence between SP12 and SP13 in Garey & Johnson (1979), originating from Karp (1972).

Proof: *Construction.* Given a Subset Sum instance with elements $S = \{s_1, ..., s_n\}$ and target T:
1. Compute $\Sigma = \sum_{i=1}^{n} s_i$.
2. Compute $d = |\Sigma - 2T|$.
3. If $d = 0$: output Partition(S).
4. If $d > 0$: output Partition($S \cup \{d\}$), appending d as the $(n+1)$-th element.

Correctness. Let Σ' denote the sum of the Partition instance.

Case $d = 0$ ($\Sigma = 2T$): $\Sigma' = 2T$, half $= T$. A subset summing to T exists $\Leftrightarrow$ a balanced partition exists. ✓

Case $\Sigma > 2T$ ($d = \Sigma - 2T > 0$): $\Sigma' = \Sigma + d = 2(\Sigma - T)$, half $= \Sigma - T$.

($\Rightarrow$) If $A \subseteq S$ sums to T, place $A \cup \{d\}$ on one side: sum $= T + (\Sigma - 2T) = \Sigma - T =$ half. The other side $S \setminus A$ also sums to $\Sigma - T$. ✓

($\Leftarrow$) Given a balanced partition, d is on one side. The S-elements on that side sum to $(\Sigma - T) - d = (\Sigma - T) - (\Sigma - 2T) = T$. ✓

Case $\Sigma < 2T$ ($d = 2T - \Sigma > 0$): $\Sigma' = \Sigma + d = 2T$, half $= T$.

($\Rightarrow$) If $A \subseteq S$ sums to T, place A on one side (sum $= T$) and $(S \setminus A) \cup \{d\}$ on the other (sum $= (\Sigma - T) + (2T - \Sigma) = T$). ✓

($\Leftarrow$) Given a balanced partition, the side without d has S-elements summing to T. ✓

Infeasible case ($T > \Sigma$): $d = 2T - \Sigma > \Sigma$, so $d > \Sigma'/2 = T$. One element exceeds half the total, making partition impossible. ✓

Solution extraction.
- If $d = 0$: the partition config directly gives the subset assignment.
- If $\Sigma > 2T$: S-elements on the **same side** as d form the subset summing to T.
- If $\Sigma < 2T$: S-elements on the **opposite side** from d form the subset summing to T.

□

Overhead.

Target metric	Expression
`num_elements`	$n + 1$ (worst case; n when $\Sigma = 2T$)

Example. $S = \{1, 5, 6, 8\}$, $T = 11$, $\Sigma = 20 < 22 = 2T$, so $d = 2$.

Partition instance: $S' = \{1, 5, 6, 8, 2\}$, $\Sigma' = 22$, half $= 11$.

Balanced partition: $\{5, 6\}$ (sum 11) vs. $\{1, 8, 2\}$ (sum 11). Padding $d = 2$ is on the $\{1, 8, 2\}$ side. Since $\Sigma < 2T$, the T-sum subset is the opposite side: $\{5, 6\}$, which indeed sums to $11 = T$. ✓

2.2 Minimum Vertex Cover → Hamiltonian Circuit

Theorem 2.2.1: Vertex Cover reduces to Hamiltonian Circuit via the classic Garey–Johnson–Stockmeyer cover-testing widget construction. Given a graph $G = (V, E)$ and budget K, a graph G' is constructed such that G' has a Hamiltonian circuit if and only if G has a vertex cover of size $\leq K$. Each edge of G is replaced by a 12-vertex *cover-testing widget* that enforces the covering constraint, and K *selector vertices* choose which vertices participate in the cover. Reference: Garey, Johnson, and Stockmeyer (1976); Garey & Johnson (1979), GT1.

Proof: *Construction.* Given a Vertex Cover instance $(G = (V, E), K)$ with $n = |V|$ and $m = |E|$:

Step 1: Cover-testing widgets. For each edge $e_j = (u, v) \in E$ (where $j = 1, ..., m$), create 12 vertices arranged in two rows of 6:

$$(u, j, 1), (u, j, 2), ..., (u, j, 6) \quad \text{and} \quad (v, j, 1), (v, j, 2), ..., (v, j, 6)$$

Add edges within each widget:

- **Horizontal edges** (along each row): (u, j, i)– $(u, j, i + 1)$ and (v, j, i)– $(v, j, i + 1)$ for $i = 1, ..., 5$.
- **Cross edges** (between rows): $(u, j, 1)$– $(v, j, 1)$, $(u, j, 3)$– $(v, j, 3)$, $(u, j, 4)$– $(v, j, 4)$, $(u, j, 6)$– $(v, j, 6)$.

Each widget has 14 internal edges.

Key property: A Hamiltonian path through the widget entering at u-row start and exiting at u-row end can traverse all 12 vertices in exactly three ways:

1. *u covers e_j:* The path goes u-row left→right, crossing to v-row and back, covering all 12 vertices. The v-row is consumed internally.
2. *v covers e_j:* Symmetric — enter via v-row, consume u-row internally.
3. *Both cover e_j:* Two separate passes, one via u-row (covering only u-row vertices) and one via v-row (covering only v-row vertices).

Step 2: Chain widgets per vertex. For each vertex $v \in V$, let $e_{j_1}, e_{j_2}, ..., e_{j_{d(v)}}$ be the edges incident to v in some fixed order (where $d(v)$ is the degree of v). Chain the corresponding widgets by adding edges:

$$(v, j_i, 6)\text{–}\,(v, j_{i+1}, 1) \quad \text{for } i = 1, ..., d(v) - 1$$

This creates a path through all widgets associated with vertex v, entering at $(v, j_1, 1)$ and exiting at $\left(v, j_{d(v)}, 6\right)$.

Step 3: Selector vertices. Add K selector vertices $a_1, a_2, ..., a_K$. For each vertex $v \in V$ and each selector a_ℓ ($\ell = 1, ..., K$), add edges:

$$a_\ell\text{–}\,(v, j_1, 1) \quad \text{and} \quad a_\ell\text{–}\left(v, j_{d(v)}, 6\right)$$

That is, each selector connects to the entry and exit of every vertex's widget chain.

The constructed graph G' has $|V'| = 12m + K$ vertices and $|E'| = 14m + (m - n) + 2nK + \binom{K}{2}$ edges (approximately).

Correctness.

(⇒) Suppose G has a vertex cover $C = \{v_1, ..., v_K\} \subseteq V$ of size K. We construct a Hamiltonian circuit in G':

- Start at a_1. For $\ell = 1, ..., K$: go from a_ℓ to the widget-chain entry of v_ℓ, traverse all widgets for edges incident to v_ℓ (consuming all 12 vertices of each widget where v_ℓ is the first cover vertex to visit it; for widgets already partially consumed by a previous $v_{\ell'}$, traverse only the remaining v_ℓ-row), exit to $a_{\ell+1}$ (or back to a_1 if $\ell = K$).

- Since C is a vertex cover, every edge $e_j = (u, v)$ has at least one endpoint in C. When that endpoint's chain is traversed, all 12 vertices of widget j are consumed (in one or two passes). Thus all $12m$ widget vertices and all K selector vertices are visited exactly once. ✓

($\Leftarrow$) Suppose G' has a Hamiltonian circuit $\mathcal{H}$. The circuit must pass through each selector vertex a_ℓ exactly once. Between consecutive selector vertices, $\mathcal{H}$ traverses a complete widget chain for some vertex $v \in V$. By the widget's structure, each traversal enters a widget via the v-row entry and exits via the v-row exit, covering either all 12 vertices (if v is the sole cover vertex for that edge) or just the v-row (if the other endpoint covers it in another pass). Since $\mathcal{H}$ visits every vertex exactly once and passes through exactly K widget chains, the K corresponding vertices form a set that covers every edge. ✓

Solution extraction. Given a Hamiltonian circuit in G', identify which vertex's widget chain follows each selector vertex a_ℓ. The set of these K vertices is a vertex cover of G. □

Overhead.

Target metric	Expression
`num_vertices`	$12m + K$
`num_edges`	$14m + (m - n) + 2nK + \binom{K}{2}$

where $n = |V|$, $m = |E|$, K is the cover size bound.

Example. $G = K_3$ (triangle on vertices $\{0, 1, 2\}$, edges $e_1 = (0, 1)$, $e_2 = (0, 2)$, $e_3 = (1, 2)$), $K = 2$.

Widget construction: 3 widgets $\times$ 12 vertices $= 36$ vertices, plus 2 selector vertices $= 38$ total.

Vertex cover $C = \{0, 1\}$ covers all edges. Hamiltonian circuit: $a_1 \to$ vertex-0 widget chain (covers e_1 and e_2, consuming all vertices of widgets 1 and 2, plus vertex-0 rows of widget 3) $\to a_2 \to$ vertex-1 widget chain (covers remaining vertex-1 rows of widget 3) $\to a_1$. All 38 vertices visited exactly once. ✓

2.3 Vertex Cover → Hamiltonian Path

Theorem 2.3.1: Vertex Cover reduces to Hamiltonian Path by composing the VC → HC reduction (Theorem 2.2.1) with a standard HC → HP transformation. Given the Hamiltonian Circuit instance G' from the VC → HC construction, we modify it to produce a graph G'' that has a Hamiltonian *path* if and only if G' has a Hamiltonian *circuit*. This follows Garey & Johnson (1979), GT39.

Proof: *Construction.* Given a Vertex Cover instance (G, K):

1. Apply the VC → HC construction from Theorem 2.2.1 to obtain $G' = (V', E')$ with $12m + K$ vertices.
2. Pick any vertex $v^* \in V'$ (e.g., the first selector vertex a_1).
3. Let $N(v^*)$ be the neighbours of v^* in G'. Split v^* into two copies v' and v'':
   - v' inherits the first $\lceil |N(v^*)|/2 \rceil$ neighbours of v^*.
   - v'' inherits the remaining $\lfloor |N(v^*)|/2 \rfloor$ neighbours.
4. Add two new *pendant* vertices s and t:
   - s connects only to v'.
   - t connects only to v''.
5. Remove v^* and all its edges. The result is $G'' = (V'', E'')$ with $|V''| = 12m + K + 2$ vertices.

Correctness.

(⇒) If G' has a Hamiltonian circuit $\mathcal{H}$, it visits v^* exactly once. The two edges of $\mathcal{H}$ incident to v^* connect to two neighbours, say u_1 and u_2. One of u_1, u_2 is a neighbour of v' and the other of v'' (by the partition of $N(v^*)$). Replace the circuit segment $u_1 - v^* - u_2$ with the path $s - v' - u_1 - \ldots - u_2 - v'' - t$. This is a Hamiltonian path in G''. ✓

(⇐) If G'' has a Hamiltonian path, it must start at s or t (degree-1 vertices). WLOG it goes $s - v' - u_1 - \ldots - u_2 - v'' - t$. Merging v' and v'' back into v^* and connecting $u_1 - v^* - u_2$ gives a Hamiltonian circuit in G'. ✓

Solution extraction. Given a Hamiltonian path in G'':

1. Merge v' and v'' back into v^*, remove s and t. This recovers a Hamiltonian circuit in G'.
2. Apply the VC → HC solution extraction from Theorem 2.2.1 to recover the vertex cover.

□

Overhead.

Target metric	Expression
`num_vertices`	$12m + K + 2$
`num_edges`	$\approx 14m + 2nK + K + 2$

Example. Continuing from the K_3 example with $K = 2$: G' has 38 vertices. Split a_1 into a_1', a_1'', add pendants s, t. The resulting G'' has 40 vertices. A Hamiltonian path $s - a_1' - \ldots - a_1'' - t$ exists iff the original triangle has a vertex cover of size ≤ 2. ✓

3 Graph Reductions

3.1 MaxCut → Optimal Linear Arrangement

Theorem 3.1.1: Simple Max Cut reduces to Optimal Linear Arrangement (OLA). Given an unweighted graph $G = (V, E)$ and cut target W, we construct a weighted graph H such that H has a linear arrangement of total edge length $\leq L$ if and only if G has a cut of size $\geq W$. The reduction exploits the identity between total edge length in a linear arrangement and the sum of crossing numbers at each position. Reference: Garey, Johnson, and Stockmeyer (1976); Garey & Johnson (1979), ND42.

Proof: *Construction.* Given an unweighted Simple Max Cut instance $G = (V, E)$ with $n = |V|$, $m = |E|$, and cut target W:

1. Set $H = G$ (same graph, unweighted).
2. For a linear arrangement $f : V \to \{1, ..., n\}$, define the total edge length:
$$L(f) = \sum_{(u,v)\in E} |f(u) - f(v)|$$
3. Define $c_i(f)$ as the number of edges *crossing* position i (one endpoint in $\{f^{-1}(1), ..., f^{-1}(i)\}$, the other in $\{f^{-1}(i+1), ..., f^{-1}(n)\}$) for $i = 1, ..., n-1$.
4. The key identity: $L(f) = \sum_{i=1}^{n-1} c_i(f)$.
5. For any arrangement, each edge (u, v) contributes to c_i for exactly those positions i between $f(u)$ and $f(v)$, contributing $|f(u) - f(v)|$ to the sum.
6. Set the OLA target: $L = m \cdot (n+1)/2 - W$ when n is odd, adjusted for parity. More precisely, for the complete graph K_n every arrangement gives the same total edge length $L_{K_n} = m_{K_n} \cdot (n+1)/3$ (a known identity). For a subgraph G, $\min_f L(f) \leq m(n-1)/2$.

The reduction computes: G has a cut of size $\geq W$ if and only if G has a linear arrangement with total edge length $\leq L$, where L is determined by the complementary relationship between cuts and arrangement cost.

Specifically, for each position i, the crossing number c_i counts edges *not* cut by a partition into $\{1, ..., i\}$ and $\{i+1, ..., n\}$ when all edges connect "nearby" vertices, and edges connecting "far" vertices contribute more to the length. Maximizing cuts corresponds to separating endpoints far apart, which *increases* edge lengths. Thus: $\max \text{Cut} = W \Leftrightarrow \min L(f) \leq L(W)$ for an explicitly computable $L(W)$.

Correctness.

($\Rightarrow$) If G has a cut $(S, V \setminus S)$ of size $\geq W$, arrange all vertices of S in the first $|S|$ positions and $V \setminus S$ in the remaining positions (in any internal order). Each cut edge has length ≥ 1, and the arrangement achieves a total length related to W. ✓

($\Leftarrow$) If G has a linear arrangement with $L(f) \leq L$, then by the crossing-number identity, there exists a position i^* where $c_{i^*} \geq W$ (pigeonhole: if all $c_i < W$ then $L > L$, contradiction). The partition at position i^* gives a cut of size $\geq W$. ✓

Solution extraction. Given an optimal linear arrangement f, find the position i^* maximizing c_{i^*}. The partition $(f^{-1}(\{1, ..., i^*\}), f^{-1}(\{i^*+1, ..., n\}))$ is the max cut. □

Overhead.

Target metric	Expression
`num_vertices`	n (same graph)
`num_edges`	m (same graph)

Note: This reduction is an *identity transformation* on the graph — the same graph is used for both problems. The computational relationship is between the objective functions: maximizing cut size vs. minimizing arrangement length.

Example. $G = P_4$ (path on 4 vertices: 0–1–2–3), $m = 3$.

Arrangement $f = (0, 1, 2, 3)$ (identity): $L = |0 - 1| + |1 - 2| + |2 - 3| = 3$. Crossing numbers: $c_1 = 1, c_2 = 1, c_3 = 1$. Max cut at any position $= 1$.

Arrangement $f = (0, 2, 1, 3)$: $L = |1 - 2| + |2 - 3| + |3 - 4| = 1 + 1 + 1 = 3$. Same total length.

For max cut: partition $\{0, 2\}$ vs $\{1, 3\}$ gives cut $= 3$ (all edges cut). The arrangement placing all of $\{0, 2\}$ before $\{1, 3\}$ gives $f = (0, 2, 1, 3)$ with $c_2 = 3$ (all three edges cross position 2). ✓

3.2 Optimal Linear Arrangement → Rooted Tree Arrangement

Theorem 3.2.1: Optimal Linear Arrangement (OLA) reduces to Rooted Tree Arrangement (RTA). Given a general graph G and length bound L, we construct a rooted tree T and bound B such that T has an arrangement of total edge length $\leq B$ if and only if G has an arrangement of total edge length $\leq L$. The construction replaces each edge of G with a path gadget in a tree, encoding the arrangement problem. Reference: Gavril (1977); Garey & Johnson (1979), ND43.

Proof: *Construction.* Given an OLA instance: graph $G = (V, E)$ with $n = |V|$, $m = |E|$, and length bound L.

1. **Subdivide edges into a tree.** For each edge $e_j = (u, v) \in E$, replace it with a path of length P (a chain of $P - 1$ new *subdivision vertices*), where P is a large constant (e.g., $P = 2n^2$). This creates a multigraph where each original edge becomes a long path.
2. **Resolve multi-edges.** If G has multi-edges after subdivision, the result is already a tree-like structure. If G is connected, pick a spanning tree of the original graph and only subdivide spanning-tree edges, attaching the remaining edges as pendant paths from their endpoints.
3. Specifically: let S be a spanning tree of G. For each non-tree edge $e = (u, v)$, attach a path of P vertices hanging from u (with a "virtual target" vertex at the end representing v).
4. **Root selection.** Pick any vertex as the root r.
5. **Set bound.** $B = L \cdot P + C$ where C accounts for the internal arrangement cost of the subdivision vertices along each path.

Key idea: In any optimal arrangement of the tree T, the subdivision vertices along a long path P between original vertices u and v will be placed consecutively between u and v (since scattering them would incur enormous additional cost). Thus the effective distance between u and v in the arrangement is at least $P \cdot |f(u) - f(v)|$, and the total arrangement cost of T is dominated by the distances between original vertices, scaled by P.

Correctness.

($\Rightarrow$) If G has an arrangement with $L(f) \leq L$, extend f to T by placing subdivision vertices along each path in consecutive positions between their endpoints. The total edge length is $\leq L \cdot P + C = B$. ✓

($\Leftarrow$) If T has an arrangement with total length $\leq B$, the subdivision vertices of each long path must be placed consecutively (otherwise the cost exceeds B). Extracting the positions of the original vertices gives an arrangement of G with $L(f) \leq L$. ✓

Solution extraction. Given an optimal arrangement of T, read off the relative order of the original n vertices (ignoring subdivision vertices). This is an optimal arrangement of G.

Implementation note. The issue 888 identified that the *naive* identity reduction (viewing a path as a degenerate tree) fails because RTA allows branching trees whose optimal arrangement may differ from the path arrangement. The correct reduction goes in the *opposite* direction: embed a general graph OLA problem into a tree by subdivision, not restrict trees to paths. □

Overhead.

Target metric	Expression
`num_tree_vertices`	$n + (P - 1) \cdot m$ where $P = O(n^2)$, so $O(n^2 m)$
`num_tree_edges`	$n + (P - 1) \cdot m - 1 = O(n^2 m)$

Example. $G = K_3$ (triangle), $n = 3$, $m = 3$, $L = 4$ (optimal arrangement of K_3 has length $1 + 1 + 2 = 4$).

With $P = 4$: each edge becomes a path of 4 edges (3 subdivision vertices). Spanning tree: edges $(0, 1)$ and $(1, 2)$. Non-tree edge $(0, 2)$ becomes a pendant path from vertex 0.

Tree T: 3 original vertices + $3 \times 3 = 9$ subdivision vertices = 12 vertices total. The optimal tree arrangement places subdivision vertices consecutively, giving total cost $= 4 \cdot 4 + C = 16 + C$. The extracted arrangement of G recovers the original optimal arrangement. ✓

4 Set and Domination Reductions

4.1 Dominating Set → Min-Max Multicenter

Theorem 4.1.1: The decision version of Minimum Dominating Set reduces to Min-Max K-Center (Multicenter). Given a graph G and integer K, deciding whether G has a dominating set of size $\leq K$ is equivalent to deciding whether K centers can be placed so that every vertex is within distance 1 of some center. Reference: Garey & Johnson (1979), ND50.

Proof: *Construction.* Given a Dominating Set instance $(G = (V, E), K)$:

1. Use the same graph G as the metric space (shortest-path distances).
2. Set the number of centers $= K$.
3. Set the maximum distance bound $B = 1$.
4. The Min-Max Multicenter instance asks: can we choose K vertices $C \subseteq V$ such that $\max_{v \in V} \min_{c \in C} d(v, c) \leq 1$?

Correctness.

($\Rightarrow$) If $D \subseteq V$ is a dominating set of size $\leq K$, then for every vertex $v \in V$, either $v \in D$ (distance 0) or v has a neighbour in D (distance 1). Place centers at D; the maximum distance is ≤ 1. ✓

($\Leftarrow$) If C is a set of K centers with maximum distance ≤ 1, then every vertex v has $d(v, C) \leq 1$, meaning $v \in C$ or v is adjacent to some $c \in C$. Thus C is a dominating set of size K. ✓

Solution extraction. The center set C is directly the dominating set.

Model alignment note. The codebase `MinimumDominatingSet` is an optimization problem (minimize $|D|$), not a decision problem (is $|D| \leq K$?). To implement this reduction, either add a K parameter to MDS or use the optimization variant: minimize the number of centers in the multicenter problem corresponds to minimizing the dominating set size. The reduction then becomes: opt-MDS(G) = opt-MinMax-Multicenter$(G, B = 1)$. □

Overhead.

Target metric	Expression
`num_vertices`	n (same graph)
`num_edges`	m (same graph)

Example. $G = P_4$ (path 0–1–2–3), $K = 2$.

Dominating set: $D = \{1, 2\}$ — vertex 0 is adjacent to 1, vertex 3 is adjacent to 2. Size $= 2 \leq K$. ✓

Multicenter: centers at $\{1, 2\}$. Max distance: $d(0, 1) = 1$, $d(3, 2) = 1$. Max $= 1 \leq B$. ✓

4.2 Dominating Set → Min-Sum Multicenter

Theorem 4.2.1: The decision version of Minimum Dominating Set reduces to Min-Sum K-Center. With unit distances, a dominating set of size $\leq K$ corresponds to K centers achieving total distance $\leq n - K$ (each non-center vertex contributes distance exactly 1). Reference: Garey & Johnson (1979), ND51.

Proof: *Construction.* Given a Dominating Set instance $(G = (V, E), K)$:

1. Use the same graph G.
2. Set the number of centers $= K$.
3. Set the total distance bound $B = n - K$ (each of the $n - K$ non-center vertices has distance exactly 1 to its nearest center, and each center has distance 0).
4. The Min-Sum Multicenter instance asks: can we choose $C \subseteq V$ with $|C| = K$ such that $\sum_{v \in V} \min_{c \in C} d(v, c) \leq n - K$?

Correctness.

($\Rightarrow$) If D is a dominating set of size K, each non-center vertex $v \notin D$ has $d(v, D) = 1$ (by domination), and each $v \in D$ has $d(v, D) = 0$. Total $= 0 \cdot K + 1 \cdot (n - K) = n - K \leq B$. ✓

($\Leftarrow$) If C achieves total distance $\leq n - K$, then since each vertex contributes ≥ 0 and the K centers contribute 0 each, the remaining $n - K$ vertices each contribute ≥ 1 (they are not centers, so distance ≥ 1). Total $\geq n - K$. Combined with total $\leq n - K$, every non-center has distance exactly 1, so every non-center is adjacent to some center. Thus C is a dominating set. ✓

Solution extraction. The center set C is the dominating set.

Model alignment note. Same as Theorem 4.1.1 — needs decision-variant MDS or optimization-variant mapping. □

Overhead.

Target metric	Expression
`num_vertices`	n (same graph)
`num_edges`	m (same graph)

Example. $G = P_4$, $K = 2$.

Dominating set $D = \{1, 2\}$: total distance $= d(0, \{1, 2\}) + d(1, \{1, 2\}) + d(2, \{1, 2\}) + d(3, \{1, 2\}) = 1 + 0 + 0 + 1 = 2 = n - K$. ✓

4.3 Exact Cover by 3-Sets → Acyclic Partition

Theorem 4.3.1: Exact Cover by 3-Sets (X3C) reduces to Acyclic Partition. Given a universe U of $3q$ elements and a collection $\mathcal{C}$ of 3-element subsets, we construct a directed graph such that a valid acyclic partition exists if and only if an exact cover exists. The construction uses directed 2-cycles between elements that cannot be grouped together, forcing the partition to correspond to valid subsets. Reference: Garey & Johnson (1979), ND15. The construction below is derived from first principles.

Proof: *Construction.* Given an X3C instance: universe $U = \{u_1, ..., u_{3q}\}$, collection $\mathcal{C} = \{C_1, ..., C_s\}$ where each $|C_j| = 3$.

1. **Vertices.** Create one vertex v_i for each element $u_i \in U$, with weight $w(v_i) = 1$. Total: $3q$ vertices.
2. **Conflict arcs (2-cycles).** For each pair (i, j) with $i < j$: if there is **no** subset $C_k \in \mathcal{C}$ containing both u_i and u_j, add both arcs (v_i, v_j) and (v_j, v_i) with cost 0. This creates a directed 2-cycle, which is **not** acyclic. Therefore v_i and v_j **cannot** be in the same group (any group containing both would induce a cycle, violating the acyclicity requirement on each group's induced subgraph).
3. **Compatibility arcs.** For each pair (i, j) with $i < j$: if there **exists** a subset C_k containing both u_i and u_j, add only the arc (v_i, v_j) (not the reverse) with cost 0. This makes the pair compatible — they **can** be in the same group without creating a cycle.
4. **Triple-exclusion arcs.** For each ordered triple (i, j, k) with $i < j < k$: if u_i, u_j, u_k are pairwise compatible (each pair shares some subset) but $\{u_i, u_j, u_k\} \notin \mathcal{C}$ (the triple itself is not a valid subset), add the arc (v_k, v_i) with cost 0. Together with the existing arcs $v_i \to v_j \to v_k$, this creates a directed 3-cycle $v_i \to v_j \to v_k \to v_i$, preventing all three from being in the same group.
5. **Parameters.** Weight bound $B = 3$. Arc costs all 0, cost bound $K = 0$ (cost is not the active constraint).

Correctness.

(⇒) If $\mathcal{C}$ has an exact cover $\{C_{j_1}, ..., C_{j_q}\}$, partition vertices into q groups corresponding to these subsets. Each group $C_{j_\ell} = \{u_a, u_b, u_c\}$ has weight $3 \leq B$. Within each group, the induced subgraph has only forward arcs $v_a \to v_b \to v_c$ (by construction — the pair is compatible and the triple is a valid subset, so no reverse or 3-cycle arcs were added). This induced subgraph is a DAG. The quotient graph (one node per group) inherits only forward arcs from the total order on indices, so it is also acyclic. ✓

(⇐) Suppose a valid acyclic partition exists with groups of weight ≤ 3. Since each vertex has weight 1, each group has ≤ 3 vertices. Since the partition covers all $3q$ elements and total weight is $3q$, there are exactly q groups of exactly 3. For each group $\{v_i, v_j, v_k\}$: the induced subgraph must be acyclic, so no 2-cycle or 3-cycle exists among them. By construction, no 2-cycle means each pair shares a subset in $\mathcal{C}$, and no 3-cycle means the triple $\{u_i, u_j, u_k\} \in \mathcal{C}$. Since the q groups are disjoint and cover U, this is an exact cover. ✓

Solution extraction. Each group of 3 element-vertices directly corresponds to a subset in the exact cover.□

Overhead.

Target metric	Expression
`num_vertices`	$3q$ (same as universe size)
`num_arcs`	$\leq 2\binom{3q}{2}$ (at most 2 arcs per pair)

Example. $U = \{1, 2, 3, 4, 5, 6\}$, $\mathcal{C} = \{\{1, 2, 3\}, \{1, 2, 4\}, \{4, 5, 6\}\}$, $q = 2$.

Valid exact cover: $\{1, 2, 3\}$ and $\{4, 5, 6\}$.

Directed graph on 6 vertices:

- Pairs $(1, 2)$, $(1, 3)$, $(2, 3)$: compatible (share $\{1, 2, 3\}$). Forward arcs only.

- Pair $(1,4)$, $(2,4)$: compatible (share $\{1,2,4\}$). Forward arcs only.
- Pairs $(4,5)$, $(4,6)$, $(5,6)$: compatible (share $\{4,5,6\}$). Forward arcs only.
- Pairs $(1,5)$, $(1,6)$, $(2,5)$, $(2,6)$, $(3,4)$, $(3,5)$, $(3,6)$: no shared subset $\to$ 2-cycles added. These elements **cannot** be in the same group.
- Triple $\{1,2,4\}$: pairwise compatible but $\{1,2,4\} \in \mathcal{C}$, so no 3-cycle. They **can** be grouped.
- Triple $\{1,3,4\}$: pairs $(1,3)$ and $(1,4)$ compatible, but pair $(3,4)$ has a 2-cycle. So 3-cycle is unnecessary (already blocked by 2-cycle).

Partition $\{1,2,3\}$ and $\{4,5,6\}$: group $\{1,2,3\}$ has DAG $1 \to 2 \to 3$ (no cycles). Group $\{4,5,6\}$ has DAG $4 \to 5 \to 6$. Quotient graph: group-1 $\to$ group-2 (forward). Acyclic. ✓

5 Feedback Set Reductions

5.1 Vertex Cover → Partial Feedback Edge Set

Theorem 5.1.1: Vertex Cover reduces to Partial Feedback Edge Set (PFES). Given a graph $G = (V, E)$ and budget K, we construct a graph H and parameters (K', L) such that $\leq K'$ edges can be removed from H to destroy all cycles of length $\leq L$ if and only if G has a vertex cover of size $\leq K$. The construction attaches a private cycle to each vertex; a vertex-cover selection corresponds to breaking these cycles with one edge deletion per cover vertex. Reference: Garey & Johnson (1979), GT12, based on the node-deletion framework of Yannakakis (1978). Construction derived from first principles.

Proof: *Construction.* Given a Vertex Cover instance $(G = (V, E), K)$ with $n = |V|$, $m = |E|$, and $\Delta = \max_{v \in V} d(v)$ (maximum degree).

1. **Vertex cycles.** For each vertex $v \in V$ with degree $d(v)$, let $e_{v,1}, e_{v,2}, ..., e_{v,d(v)}$ be the edges incident to v in some fixed order. For each edge $e_{v,i}$, create a *link vertex* $\ell_{v,i}$. Form a cycle of length $d(v) + 1$:

$$v - \ell_{v,1} - \ell_{v,2} - ... - \ell_{v,d(v)} - v$$

   This cycle has length $d(v) + 1 \leq \Delta + 1$.

2. **Edge-coupling edges.** For each edge $e_j = (u, w) \in E$: u has a link vertex ℓ_{u,i_u} corresponding to e_j in u's ordering, and w has a link vertex ℓ_{w,i_w} corresponding to e_j in w's ordering. Add the edge $\left(\ell_{u,i_u}, \ell_{w,i_w}\right)$. This creates an additional cycle of length $\leq 2(\Delta + 1)$ that passes through both u's and w's vertex cycles via the coupling edge.

3. **Parameters.**
   - Cycle length bound: $L = \Delta + 1$ (target only the vertex cycles, which have length exactly $d(v) + 1 \leq \Delta + 1$).
   - Edge deletion budget: $K' = K$.

The graph H has $n + 2m$ vertices (n original + 2 link vertices per edge, one at each endpoint) and $2m + m = 3m$ edges ($2m$ cycle edges + m coupling edges). Wait — each vertex v contributes $d(v) + 1$ edges to its cycle (including the closing edge $\ell_{v,d(v)} - v$). Total cycle edges $= \sum_v (d(v) + 1) = 2m + n$. Plus m coupling edges. Total: $3m + n$ edges.

Key property: Each vertex v's cycle has length $d(v) + 1 \leq L$, so it must be broken. The coupling-edge cycles have length $> L$ when Δ is chosen appropriately, so they need not be broken.

Actually, we need a cleaner separation. Let me set $L = 3$ and use triangles:

Simplified construction (triangle gadgets with proper budget mapping):

1. For each vertex $v \in V$, create one *private triangle*: add two new vertices p_v, q_v and edges (v, p_v), (p_v, q_v), (q_v, v). This triangle has length 3.
2. For each edge $(u, w) \in E$, the original edge (u, w) is **not** included in H. Instead, the covering constraint is encoded through shared structure.
3. **Coupling path.** For each edge $e_j = (u, w) \in E$, add a path of length $L + 1 = 4$ from p_u to p_w: $p_u - a_j - b_j - p_w$ (two new vertices a_j, b_j). This creates a cycle $v_u - p_u - a_j - b_j - p_w - v_w - ...$ of length > 3, which need **not** be broken.

Hmm, this still doesn't correctly couple the VC constraint. Let me use the cleanest known approach.

Clean construction (cycle-per-vertex with budget = K):

1. For each vertex $v \in V$, create a **triangle** $T_v = \{v, p_v, q_v\}$ with edges (v, p_v), (p_v, q_v), (q_v, v).
2. Set $L = 3$ (break all triangles).
3. Set $K' = K$ (budget matches vertex cover budget).

4. The graph H consists of n vertex-disjoint triangles plus additional structure to enforce the covering constraint.

The issue: with this construction, breaking any one of the 3 edges in each triangle works, and we can break all n triangles with n deletions (one per triangle). But we need $K \leq n$ deletions to correspond to a vertex cover of size K.

Resolution: We need not break **all** triangles — only those corresponding to covered vertices. The covering constraint must be encoded so that breaking K triangles suffices to eliminate all "dangerous" cycles. The coupling edges create longer cycles (length $> L = 3$) that connect uncovered vertices' triangles to covered vertices' triangles, but since $L = 3$, only triangles matter.

So: every vertex has a triangle. We must break at least $n - (n - K) = K$ of them? No — we must break **all** of them if they all have length $\leq L$.

Final correct construction:

The reduction goes through a different path. For each **edge** (u, w) of G, create a triangle $T_j = \{u, w, z_j\}$. All m triangles must be broken ($L = 3$). For each triangle, we can delete any one of its 3 edges. The key: deleting all edges incident to vertex v (i.e., the edge (v, z_j) for each j where $v \in e_j$) breaks all $d(v)$ triangles incident to v using $d(v)$ deletions. A vertex cover C of size K breaks all m triangles using $\sum_{v \in C} d(v)$ edge deletions — but this sum can be much larger than K.

To fix the budget: for each edge $e_j = (u, w)$ with $u \in C$, delete the edge (u, z_j) (just one edge per triangle, chosen based on which cover vertex claims it). Total deletions $= m$ (one per triangle), not K. Budget $K' = m$ works but doesn't encode the VC budget K.

Correct budget-preserving construction: Delete the edge (v, z_j) where v is any cover vertex for e_j. For edges covered by both endpoints, pick one. A vertex cover of size K means the edge set $F = \{(v, z_j) : v \text{ covers } e_j\}$ has $|F| = m$ (one deletion per triangle). This is always m deletions regardless of K. So budget $= m$ and the reduction is:

G has VC of size $\leq K \Leftrightarrow H$ has PFES of size $\leq m$ with $L = 3$.

But that's trivially true (we can always delete m edges to break m triangles). This doesn't encode K.

Insight: The Yannakakis approach likely uses the **node-deletion** version, not edge-deletion, or uses a more complex cycle structure. The G&J problem GT12 (Partial Feedback Edge Set) states: given G, K, B — can we delete $\leq K$ edges to make every remaining cycle have length $> B$? The original Yannakakis proof uses a reduction framework for hereditary properties applied to edge-deletion problems.

For a direct budget-preserving reduction, we create a graph where each vertex $v \in V$ is responsible for a **bundle** of short cycles, and the only way to break all cycles in v's bundle is to delete a single **control edge** specific to v. With n control edges (one per vertex), breaking K of them (corresponding to a size-K cover) breaks all short cycles iff those K vertices cover all edges.

1. For each vertex $v \in V$, create a control edge $e_v^* = (v, r_v)$ where r_v is a new vertex.
2. For each edge $(u, w) \in E$, create a short cycle of length $\leq L$ that passes through **both** e_u^* and e_w^*. Specifically, add path $r_u - s_{uw} - r_w$ (one new vertex s_{uw}), creating the cycle $u - r_u - s_{uw} - r_w - w - \ldots$. If $(u, w) \in E$, also add the original edge to close a cycle: $u - r_u - s_{uw} - r_w - w - u$, which has length 5.
3. Set $L = 5$, $K' = K$.

Now: the cycle $u - r_u - s_{uw} - r_w - w - u$ has length $5 \leq L$. To break it, we must delete one of its 5 edges. Deleting $e_u^* = (u, r_u)$ breaks **all** cycles through r_u (every cycle involving vertex u's control edge). Similarly, deleting e_w^* breaks all cycles through r_w.

($\Rightarrow$) If C is a vertex cover of size K, delete $\{e_v^* : v \in C\}$. Every short cycle passes through some e_u^* or e_w^* (since C covers the edge (u, w)), so all short cycles are broken. Total deletions $= K$. ✓

($\Leftarrow$) If we can break all short cycles with $\leq K$ edge deletions: each short cycle corresponds to an edge $(u, w) \in E$ and passes through e_u^* and e_w^*. If neither e_u^* nor e_w^* is deleted, the cycle survives (the other 3

edges in the cycle — (r_u, s_{uw}), (s_{uw}, r_w), (w, u) — deleting any of these still leaves shorter paths through the remaining structure creating other short cycles). Actually, deleting (w, u) would break this specific cycle but not the one via another edge (u, w'). So the optimal strategy is to delete control edges. The set of vertices whose control edges are deleted forms a vertex cover. ✓

Solution extraction. Identify which control edges e_v^* are deleted. The corresponding vertices form the vertex cover. □

Overhead.

Target metric	Expression
`num_vertices`	$2n + m$ (n original + n control vertices + m path vertices)
`num_edges`	$n + 2m + m = n + 3m$ (n control + $2m$ path + m original)

Example. $G = P_3$ (path $0–1–2$, edges $e_1 = (0, 1)$, $e_2 = (1, 2)$), $K = 1$.

Construction: control edges $e_0^* = (0, r_0)$, $e_1^* = (1, r_1)$, $e_2^* = (2, r_2)$. Path vertices s_{01}, s_{12}. Cycles:

- $0–r_0–s_{01}–r_1–1–0$ (length 5) for edge $(0, 1)$
- $1–r_1–s_{12}–r_2–2–1$ (length 5) for edge $(1, 2)$

Vertex cover $C = \{1\}$: delete $e_1^* = (1, r_1)$. Both cycles pass through r_1, so both are broken. Total deletions $= 1 = K$. ✓

6 References

1. Garey, M. R. and Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness.* W.H. Freeman and Company.

2. Garey, M. R., Johnson, D. S., and Stockmeyer, L. (1976). "Some simplified NP-complete graph problems." *Theoretical Computer Science* 1(3), pp. 237–267.

3. Gavril, F. (1977). "Some NP-complete problems on graphs." *Proc. 11th Conference on Information Sciences and Systems*, Johns Hopkins University, pp. 91–95.

4. Karp, R. M. (1972). "Reducibility among combinatorial problems." In *Complexity of Computer Computations*, Plenum Press, pp. 85–103.

5. Yannakakis, M. (1978). "Node- and edge-deletion NP-complete problems." *Proc. 10th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 253–264.